



UNIVERSIDADE FEDERAL DA PARAÍBA – CIÊNCIA DE DADOS PARA NEGÓCIOS
PROJETO ORIENTADO À OBJETOS

MR

Muscle Rank

SAÚDE E DIVERSÃO

***UM APP DE
GESTÃO DE
TREINOS
GAMEFICADO***



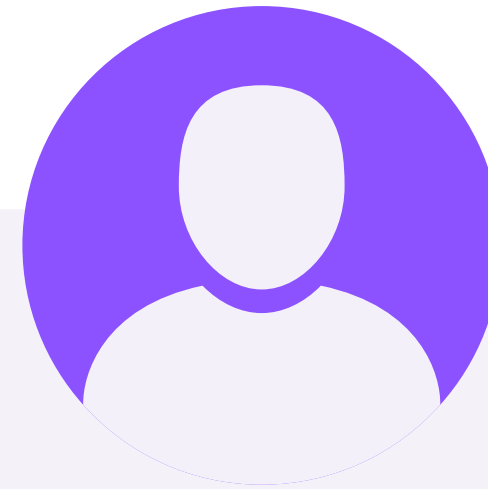
GRUPO:



ALEX
TAVARES



ARTHUR
SILVA



KAIO
VICTOR

PROFESSORA: **ADRIANA**
DAMASCENO



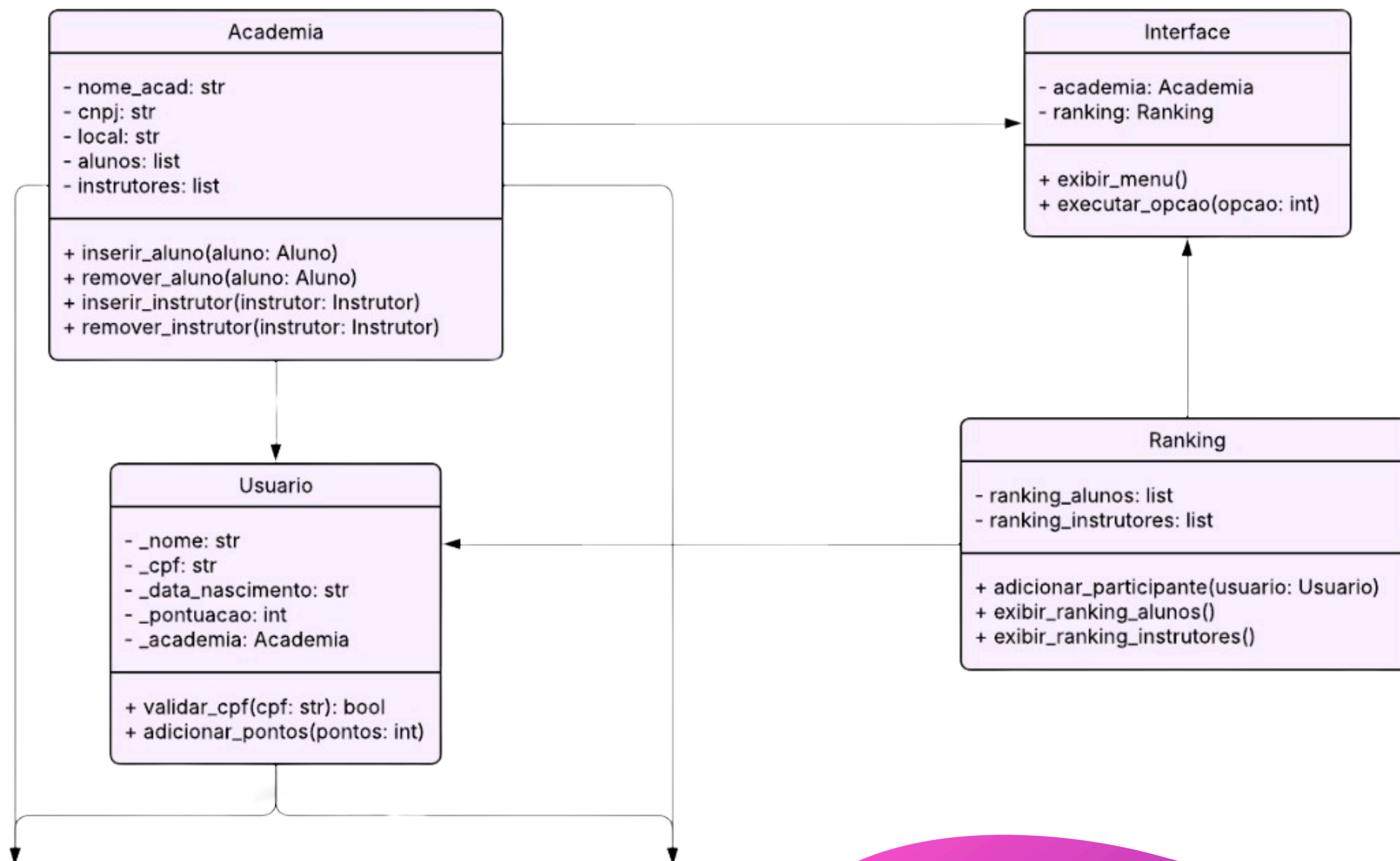
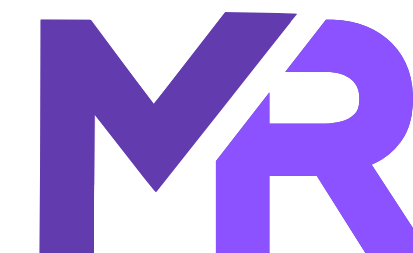
Já pensou em deixar seu treino mais organizado e ao mesmo tempo mais divertido?

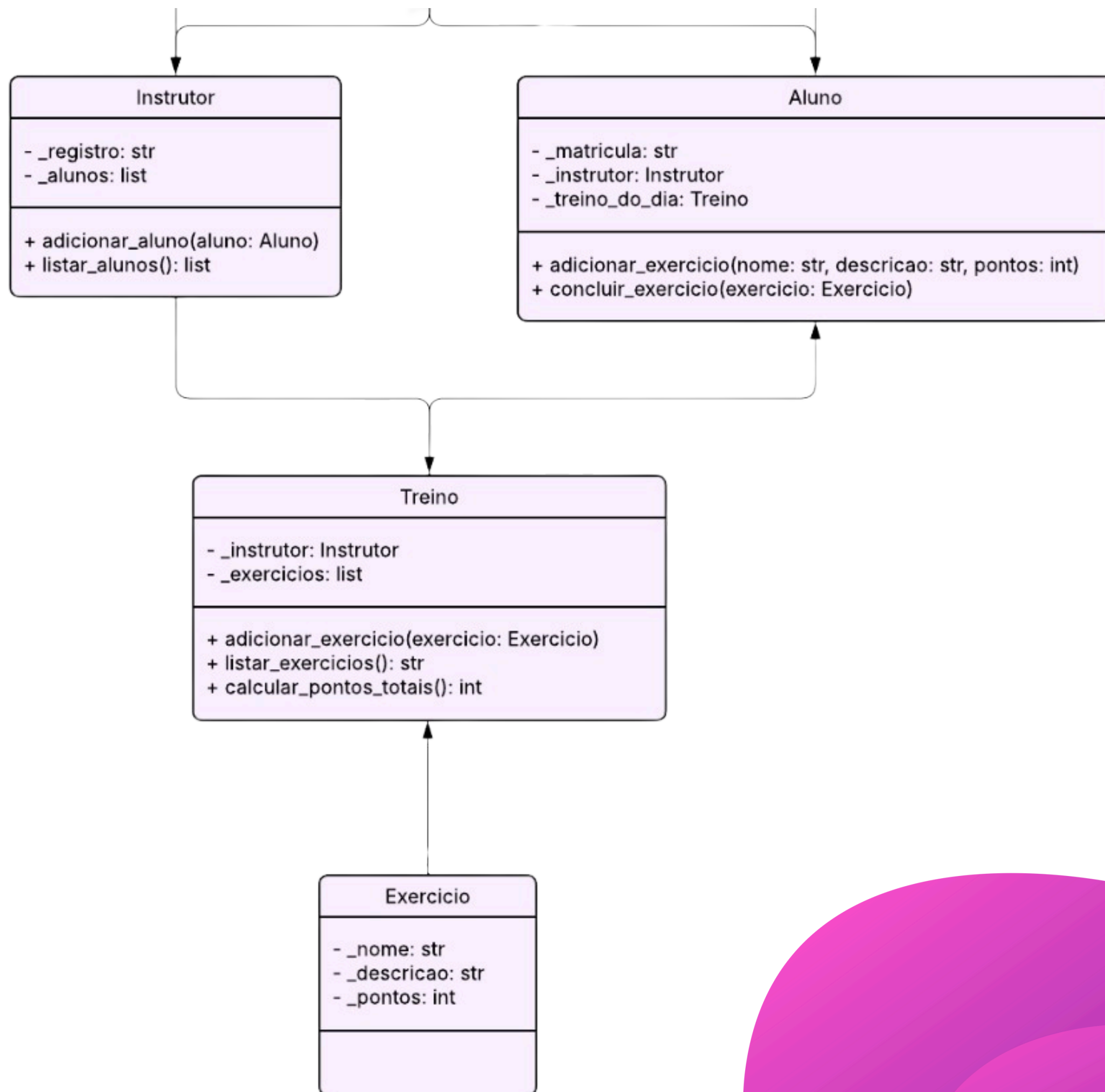
Nós somos a **Muscle Rank**, uma ferramenta dedicada a ajudar academias a terem alunos mais ativos e praticantes, tornando a atividade física mais divertida e competitiva. Através da gamificação, os alunos acumulam pontos ao realizar exercícios e treinos, competindo entre si com base em seus interesses e objetivos, enquanto disputam por recompensas motivadoras. Dessa maneira, estimulamos o engajamento e a constância nos treinos, transformando o hábito de se exercitar em uma experiência envolvente e desafiadora.

Serviços Oferecidos:

1. Adicionar Aluno
2. Adicionar Instrutor
3. Adicionar Exercício ao Treino do Dia
4. Concluir Exercício
5. Deletar treino
6. Exibir Ranking de Alunos
7. Exibir Ranking de Instrutores
8. Exibir Informações da Academia
9. Exibir Alunos da Academia
10. Exibir Instrutores da Academia
11. SAIR







Atualizações:

- Diagrama de classes
- Construtores
- Destrutores
- Herança
- Composição
- Sobrecarga de operadores
- Polimorfismo
- Encapsulamento (usando decoradores)
- Métodos de instância
- Tratamento de exceções.



Herança:

Aluno e Instrutor herdam de Usuário.

```
1 class Aluno(Usuario):  
2     def __init__(self, nome, cpf, data_nascimento, matricula, instrutor, academia):  
3         super().__init__(nome, cpf, data_nascimento, academia)
```

```
1 class Instrutor(Usuario):  
2     def __init__(self, nome, cpf, data_nascimento, registro, academia):  
3         super().__init__(nome, cpf, data_nascimento, academia)
```


Composição:

- Usuario possui um atributo que é uma instância da classe Academia.
- Treino contém Exercicio em uma lista de exercícios.
- Aluno tem um Instrutor associado.



```
1 class Usuario:
2     def __init__(self, nome, cpf, data_nascimento, academia):
```



```
1 class Treino:
2     def __init__(self, instrutor):
3         self._instrutor = instrutor
4         self._exercicios = []
5         self._data_criacao = datetime.now()
```

Sobrecarga de Operadores:

Utiliza o operador `__len__` para impedir que o usuário adicione mais que 10 exercícios ao seu treino do dia.



```
1 class Treino:
2     def __len__(self):
3         return len(self._exercicios)
```



```
1 class Interface:
2     if len(aluno.treino_do_dia) ≥ 10:
3         print("O treino já atingiu o limite de 10 exercícios!")
```

Polimorfismo:

O método `__str__` é sobrescrito em Aluno, Instrutor, Treino e Exercício.

```
1 class Aluno:
2     def __str__(self):
3         return f"Aluno: {self.nome} - Pontuação: {self.pontuacao}"
```

```
1 class Instrutor:
2     def __str__(self):
3         return f"Instrutor: {self.nome} - Pontuação: {self.pontuacao}"
```

```
1 class Exercicio:
2     def __str__(self):
3         return (f"\nNome: {self._nome}"
4                 f"\nDescrição: {self._descricao}"
5                 f"\nPontos: {self._pontos}")
```

```
1 class Treino:
2     def __str__(self):
3         return (f"\nInstrutor: {self._instrutor}"
4                 f"\nCriado em: {self._data_criacao.strftime('%d/%m/%Y %H:%M')}"
5                 f"\n\nExercícios: {self.listar_exercicios()}"
6                 f"\n\nPontos Totais: {self.calcular_pontos_totais()}")
```


Encapsulamento:

Atributos de todas as classes principais (menos interface).

```
1 class Academia:
2     def __init__(self, nome_acad, cnpj, local):
3         self.__nome_acad = nome_acad
4         self.__cnpj = cnpj
5         self.__local = local
6         self.alunos = []
7         self.instrutores = []
8
```

Tratamento de Exceções:

Validar o CPF do usuário, instrutor não registrado, verificar se as instâncias de alguns objetos estão corretas.

```
1 class Usuario:  
2     def __init__(self, nome, cpf, data_nascimento, academia):  
3         if not self.validar_cpf(cpf):  
4             raise ValueError("CPF inválido!")  
5  
6     @staticmethod  
7     def validar_cpf(cpf):  
8         #verifica se possui 11 digitos  
9         return isinstance(cpf, str) and cpf.isdigit() and len(cpf) == 11
```

Destrutores:

Remove todos os alunos após encerrar o programa.

```
1 def __del__(self):  
2     print(f"Aluno {self.nome} foi removido do sistema.")  
3     self.instrutor = None  
4     self._treino_do_dia = None
```

11. SAIR

Escolha uma opção: 11

Saindo do sistema...

Aluno Ana Souza foi removido do sistema.

Aluno João Lima foi removido do sistema.

Aluno alex foi removido do sistema.

Conclusão dos Métodos Aplicados:

O uso desses conceitos de POO garante um código organizado, reutilizável e seguro. O destrutor libera memória ao remover alunos ao encerrar o programa, enquanto a herança reutiliza código ao estruturar Aluno e Instrutor como subclasses de Usuário. A composição melhora a modularidade ao definir relações entre classes, como Aluno e Instrutor ou Treino e Exercício.

A sobrecarga de operadores impõe restrições ao treino diário, reforçando regras de negócio. O polimorfismo melhora a legibilidade ao personalizar representações textuais dos objetos. O encapsulamento protege atributos sensíveis, e o tratamento de exceções evita erros, garantindo um sistema robusto.

Dúvidas e Dificuldades:

Durante o projeto encontramos...





Obrigado pela
atenção!

[Clique aqui para obter o código!](#)